MASS: Mutation-based Assessment of Solidity Static Analyzers

SE-801 SPL-3 Technical Report

Supervised by
Adwait Nadkarni
Associate Professor
Computer Science Department,
Williams & Mary
Williamsburg, Virginia

Mentored by
Amit Seal Ami
PhD. Candidate
Computer Science Department,
Williams & Mary
Williamsburg, Virginia

Internally Supervised by
Dr. Md. Mahbubul Alam Joarder
Professor
Institute of Information Technology,
University of Dhaka

Submitted by
Shazzad Hossain
BSSE-1203
Institute of Information Technology,
University of Dhaka

Submission Date: December 1, 2024

Letter of Transmittal

November 30, 2024
BSSE 4th Year Exam Committee
Institute of Information Technology,
University of Dhaka

Subject: **Technical Report Submission - MASS: Mutation-based Assessment of Solidity Static Analyzers.**

Sir,

I am submitting the technical report for our Software Project Lab 3, "**MASS**: Mutation-based Assessment of Solidity Static Analyzers".

This report comprehensively covers the project's technical details, including design, implementation, and testing phases. While I've tried to do my best, I welcome your feedback for improvement.

Thank you for your consideration.

Sincerely,
Shazzad Hossain
BSSE-1203
Institute of Information Technology
University of Dhaka

01.12.2024
_____
Supervisor's Signature

Acknowledgement

I am grateful to the Institute of Information Technology for giving me the opportunity to work on the project “MASS: A Tool for Evaluating Solidity Static Analysis Tools using Mutation based Bug Injection”. I would like to convey my tremendous gratitude to my supervisors, Dr. Md. Mahbubul Alam Joarder, Professor, IIT, and Adwait Nadkarni, Associate Professor, W&M, and my mentor Amit Seal Ami, PhD. candidate, W&M, for providing me with guidance and support throughout the project. Their expertise and insights were instrumental in shaping the direction of the project. Lastly, I would like to thank my classmates. They have always been helpful and provided valuable insights from time to time.

Abstract

Smart contracts are the building blocks of business logic of blockchain-based systems, enabling secure and autonomous transactions. However, any exploitation in security flaws in smart contracts can result in significant financial and reputational losses. Since smart contracts are immutable, ensuring the security of these contracts before deployment is therefore crucial.

MASS (Mutation-based Assessment of Solidity Static Analyzers) is developed to evaluate the effectiveness of static analysis tools in detecting vulnerabilities within Solidity smart contracts. MASS employs a bug injection approach in which security-relevant bugs are injected into smart contracts, replicating real-world vulnerabilities. This approach allows for a systematic evaluation of static analysis tools by uncovering both false negatives—vulnerabilities that the tools fail to identify—and false positives—instances where harmless code is incorrectly flagged as vulnerability. By introducing these controlled defects, MASS provides insights into the strengths and weaknesses of various static analysis tools. These insights enable developers and security researchers to enhance the reliability of smart contract security audits, ultimately contributing to the development of more secure and trustworthy blockchain applications.

Table of Contents

1. Introduction **8**
1.1 Motivation 8
1.2 Scope 9
1.3 Project Timeline 10
1.4 Purpose 11
2. Project Description **12**
2.1 Background 12
2.1.1 Static Code Analyzers 12
2.1.2 Tools to be Evaluated 13
2.1.2.1 Slither 13
2.1.2.2 Aderyn 13
2.1.2.3 Mythril 13
2.1.3 Common Security Vulnerabilities in Smart Contract 14
2.2 Quality Function Deployment 14
Normal Requirements 15
Expected Requirements 15
Exciting Requirements 15
2.3 Usage Scenario 15
A. Initial Setup 15
B. Customizing Vulnerability Injection 15
C. Executing the Mutation Engine 16
D. Automatic Evaluation with Predefined Tools 16
E. Viewing Reports and Analysis 16
3. Scenario-based Modeling **17**
3.1 Use Case Diagram 17
Level 0: MASS 17
Level 1: MASS 17
Description of Use Case Diagram Level 1: 18
4. Data-based Modeling **19**
5. Class-based Modeling **20**
5.1 Analysis Classes 20
5.2 Class Cards 21
5.3 Class-Responsibility-Collaborator Diagram 25
6. Software Design Architecture **26**
6.1 Architectural Context Design 26

6.2 Top Level Components....27
6.3 Instantiation of the Architecture....27
Component: Mutation....28
Component: Evaluation....28
6.4 Mapping Requirements to Software Architecture....29
7. Preliminary Test Plan....30
7.1 High-Level Description of Testing Goals....30
7.2 Test Cases....30
Test Case 1: Mutation Syntactic Validity....30
Test Case 2: Mutation Semantic Validity....31
Test Case 3: Static Analysis Tool Integration....31
Test Case 4: Handling Analysis Tool Failures....32
Test Case 5: Detection of False Negatives....32
Test Case 6: Detection of False Positives....32
Test Case 7: Report Accuracy Verification....33
Test Case 8: Handling of Valid User Configurations....33
Test Case 9: Handling of Invalid User Inputs....34
Test Case 10: Performance Under Heavy Load....34
Test Case 11: Scalability Testing with Increasing Workloads....35
8. Conclusion....36
References....37

List of Figures

Figure 1: Timeline for MASS....10
Figure 2: Use Case Diagram Level 0....16
Figure 3: Use Case Diagram Level 1....17
Figure 4: CRC Diagram of MASS....24
Figure 5: Architectural Context Diagram of MASS....25
Figure 6: Top Level Components of MASS....26
Figure 7: Architecture Instantiation on Component 'Mutation'....27
Figure 8: Architecture Instantiation on Component 'Evaluation'....27

List of Tables

Table 1: CRC Card of ContractParser....20
Table 2: CRC Card of AST....20
Table 3: CRC Card of Configuration....21
Table 4: CRC Card of MutationEngine....21
Table 5: CRC Card of VulnerabilityInjectionEngine....22
Table 6: CRC Card of ToolRunner....22
Table 7: CRC Card of ResultInspector....23
Table 8: CRC Card of ReportGenerator....23
Table 9: Mapping of Components with Requirements....28

1. Introduction

In the realm of blockchain technology, the security of smart contracts is of utmost importance. Solidity, the programming language for Ethereum-based smart contracts, is susceptible to a variety of vulnerabilities that can lead to significant financial losses and undermine user trust. To safeguard against these risks, static analysis tools are employed to examine smart contracts for potential security flaws before they are deployed.

Mutation-based **A**ssessment of **S**olidity **S**tatic **A**nalyzers **(MASS)** is designed to systematically evaluate the effectiveness of these static analysis tools in detecting vulnerabilities within Solidity smart contracts. MASS leverages a mutation engine that strategically injects security-relevant bugs into smart contracts, simulating real-world vulnerabilities. By introducing these controlled defects, MASS allows for a detailed assessment of static analysis tools, identifying both false negatives—bugs the tools fail to detect—and false positives—incorrectly flagged issues. The aim is to provide developers and security researchers with valuable insights into the strengths and weaknesses of various static analysis tools, enhancing the reliability of smart contract security auditing processes.

This evaluation process provides developers and security researchers with valuable insights into the strengths and limitations of some static analysis tools. By highlighting the capabilities and shortcomings of these tools, MASS enhances the reliability of smart contract security auditing processes.

1.1 Motivation

Smart contracts are a foundational element of blockchain technology, enabling decentralized applications to execute transactions and enforce agreements autonomously. Their ability to handle real-world assets and facilitate trustless interactions underscores their transformative potential across various industries. However, this potential is accompanied by significant challenges related to reliability and security. The immutable nature of deployed smart contracts means that any undetected vulnerabilities can lead to severe financial losses, compromised assets, and erosion of user trust. Given that smart

contracts often manage substantial value, the stakes for ensuring their security are exceptionally high.

Despite the critical role smart contracts play, the effectiveness of existing static analysis tools for Solidity—the primary language for Ethereum smart contracts—remains inconsistent. These tools are prone to generating false positives, where benign code is incorrectly flagged as vulnerable, and false negatives, where actual vulnerabilities go undetected. Such inaccuracies undermine the reliability of security assessments, leaving smart contracts exposed to potential exploits and attacks. The lack of dependable static analysis results necessitates a more systematic evaluation approach to ascertain the true effectiveness of these tools.

In order to address this gap, MASS (Mutation-based Assessment of Solidity Static Analyzers) is designed for evaluating the performance of static analysis tools used in smart contract security. The primary motivations behind developing MASS are twofold:

- Development of a security-focused mutation engine that can inject vulnerabilities into smart contracts [1] and
- Comprehensive evaluation of the performance of existing static analysis tools by analyzing the reported false-negatives and false-positives [2].

The evaluation will provide insights into the accuracy, efficiency, and limitations of these tools.

1.2 Scope

The scope of the project is to -

- Take solidity smart contracts
- Take the configuration of the tests i.e., which vulnerabilities to be injected and which tool(s) should be used to analyze the contract
- Generate Abstract Syntax Tree (AST) of the smart contract

- Identify the the potential locations for injecting bugs and generate an annotated AST marking the identified locations
- Inject bugs into the marked locations to generate the buggy contract
- Check the buggy contract using the selected tools and inspect the results for undetected bugs and false alarms
- Generate reports of the false positives and the false negatives, and the time taken to analyze the contract by each tool.

1.3 Project Timeline

This chapter describes the project timeline, which contains a detailed breakdown of the major phases, tasks, and timelines that will guide the project from inception to completion. The major phases are —

- Gather prerequisite knowledge
- SRS report documentation
- Designing the complete project
- Development and Testing
- Deployment of the final project
- Final report writing

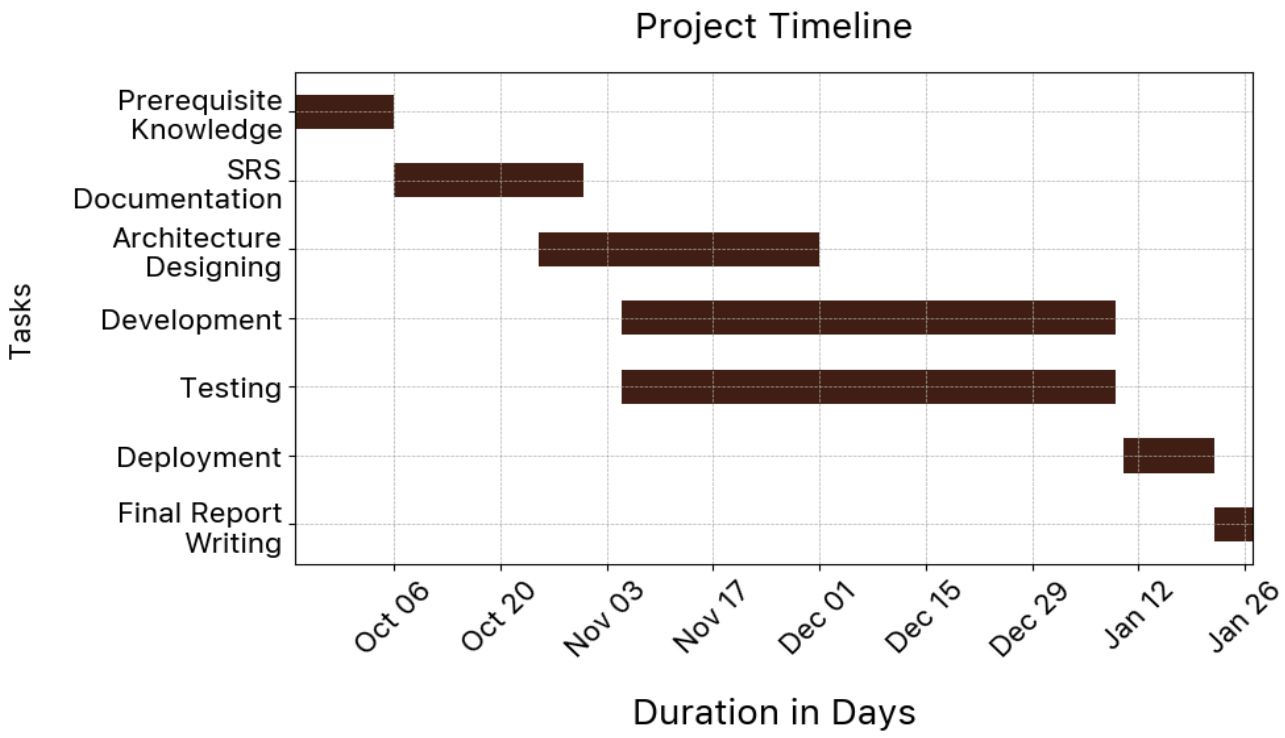


Fig 1: Timeline for MASS.

1.4 Purpose

The purpose of this document is to -

- Identify and analyze the requirements
- Design the architecture
- Describe methodology
- Design the test plan
- Reduce the development effort
- Improve understanding

2. Project Description

The following depicts the overview of the quality function deployment of MASS. The journey toward defining MASS' Quality Function Deployment (QFD) began with an in-depth review of relevant research literature, notably, the paper authored by Asem and Karthik [1], providing valuable insights into the domain.

For this project, I have identified the smart contract developer community as the stakeholders. Thus I have conducted several interviews with some of the developers currently working in the industry to gain insight into their views on the issue of SAST tools and common vulnerabilities. After these discussions, I have determined some preliminary set of solution requirements for this project.

This chapter presents the Background of the project, Quality function deployment and the usage scenario of the tool.

2.1 Background

There are two types of analyzers for programming languages — Static and Dynamic Analyzers. The goal of this project is to systematically evaluate a set of Static analysis tools for Solidity programming language.

2.1.1 Static Code Analyzers

- **Definition:** Tools that examine code without executing it, focusing on the structure, syntax, and logic of the program. They typically scan the source code, bytecode, or intermediate code to find issues like coding standards violations, potential security vulnerabilities, or logical errors.
- **Focus:** Analyze code at an early stage, providing feedback on issues such as code quality, security vulnerabilities, and adherence to coding standards.
- **Advantages:**

  → Can be automated, enabling frequent checks throughout development.

→ Detects errors early in the development process before the code is run.

→ Requires less computational resources.

2.1.2 Tools to be Evaluated

2.1.2.1 Slither

Slither is a popular open-source static analysis framework developed by Trail of Bits specifically designed for Solidity code. It provides a suite of automated vulnerability detectors that can identify common issues such as reentrancy, unused variables, and authorization vulnerabilities. Slither's efficiency lies in its ability to analyze contracts quickly, presenting detailed findings and offering recommendations on code improvements. Its output is compatible with various reporting formats, making it suitable for comprehensive vulnerability detection in larger-scale projects. [3]

2.1.2.2 Aderyn

Aderyn is a newer tool for static analysis that aims to simplify the detection and reporting of specific vulnerabilities in Solidity contracts. It utilizes a report-based approach, producing structured outputs that are easy to parse and interpret. Aderyn's focus is on enhancing the reporting aspect of vulnerability analysis, enabling a clearer understanding of potential risks in smart contracts. Its structured format makes it a complementary tool for analyzing vulnerabilities identified by other platforms like Slither, as it can help further document and categorize detected issues. [4]

2.1.2.3 Mythril

Mythril is a widely-used, free security tool created by ConsenSys Diligence specifically for Ethereum smart contracts. It helps developers find a variety of security issues, such as reentrancy attacks, integer overflows, and problems with access controls. By using advanced techniques, Mythril can uncover hidden bugs that other tools might miss, ensuring that smart contracts are more secure and reliable. Its user-friendly command-line interface makes it easy to integrate Mythril into existing development processes and automated testing pipelines. [8]

2.1.3 Common Security Vulnerabilities in Smart Contract

Some common security vulnerabilities in smart contracts are discussed below. [5]

- ***Reentrancy Attacks.*** One of the most common and dangerous vulnerabilities in Solidity. It occurs when a contract makes an external call to another untrusted contract before updating its own state, allowing malicious contracts to call back and exploit the logic.

- ***Unchecked External Calls.*** External calls (e.g., to other contracts) might fail and return false. Not checking return values can lead to failed Ether transfers or broken functionality.

- ***Timestamp Dependency.*** The current timestamp of the block can be used by contracts to trigger some time-dependent events. Given the decentralized nature of Ethereum, miners can manipulate the timestamp (to some extent). Malicious miners can use this capability and change the timestamp to favor themselves. Therefore, developers should not rely on the precision of the block's timestamp.

- ***Use of tx.origin.*** In a chain of calls, when contracts call functions of each other, the use of *tx.origin* (that returns the first caller that originally sent the call) for authentication instead of *msg.sender* (that returns the immediate caller) can lead to phishing-like attacks.

- ***Unchecked Send.*** Unauthorized Ether transfer, such as non-zero sends, can be called by external users if they are visible to the public, even if they do not have the correct credentials. This means unauthorized users can call such functions and transfer ether from the vulnerable contract.

2.2 Quality Function Deployment

Quality Function Deployment (QFD) is a technique that translates the needs of the customer into technical requirements for the software. The following requirements have been identified for this project.

Normal Requirements

NR1 - Users can configure MASS to select which bugs to inject into the smart contracts.

NR2 - Users can view the report generated by MASS after analyzing the contracts with static analysis tools.

Expected Requirements

ER1 - MASS will automatically analyze the source code, inject bugs and analyze the contracts with the static analysis tools.

ER2 - Report will contain false positives and false negatives reported by each of the tools.

Exciting Requirements

Exc1 - Report will contain comparative analysis of performance metrics of the tools.

Exc2 - Users could receive insights into how injected vulnerabilities impact gas costs.

2.3 Usage Scenario

A. Initial Setup

The user accesses the MASS platform on their local environment or server. They upload one or more Solidity smart contracts, or a directory containing the contracts that they want to analyze.

B. Customizing Vulnerability Injection

The user specifies the types of vulnerabilities to inject into the contracts. For example, they may choose options like reentrancy, unchecked send etc.

C. Executing the Mutation Engine

The user initiates the mutation process, and MASS parses the smart contracts into Abstract Syntax Trees (ASTs) to identify syntactically valid and semantically appropriate locations for injecting the chosen vulnerabilities.

D. Automatic Evaluation with Predefined Tools

MASS automatically runs the predefined static analysis tools on the mutated contracts. Each tool's results, including detected vulnerabilities, false positives, and false negatives, are available as markdown files.

E. Viewing Reports and Analysis

Once the analysis is complete, the user accesses the report. The report will contain a list of vulnerabilities detected and missed by each tool, highlighting both false positives and false negatives, and comparative performance metrics such as detection rate and the time each tool took to complete the analysis.

3. Scenario-based Modeling

This chapter describes the scenario-based modeling for MASS.

3.1 Use Case Diagram

A use case is a list of actions or event steps typically defying the interactions between a role (actor) and a system to achieve a goal. Use case diagrams of MASS are given below.

Level 0: MASS

Name: MASS

Primary Actor: User

Secondary Actor: Static Analysis Tools

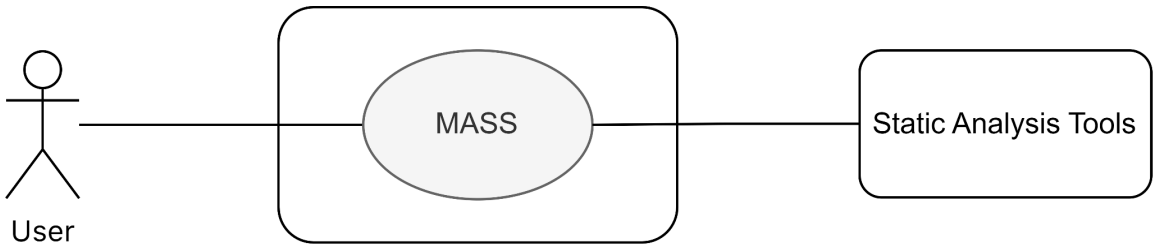


Fig 2: Use Case Diagram Level 0.

Level 1: MASS

Name: Modules of MASS

Primary Actor: User

Secondary Actor: Static Analysis Tools

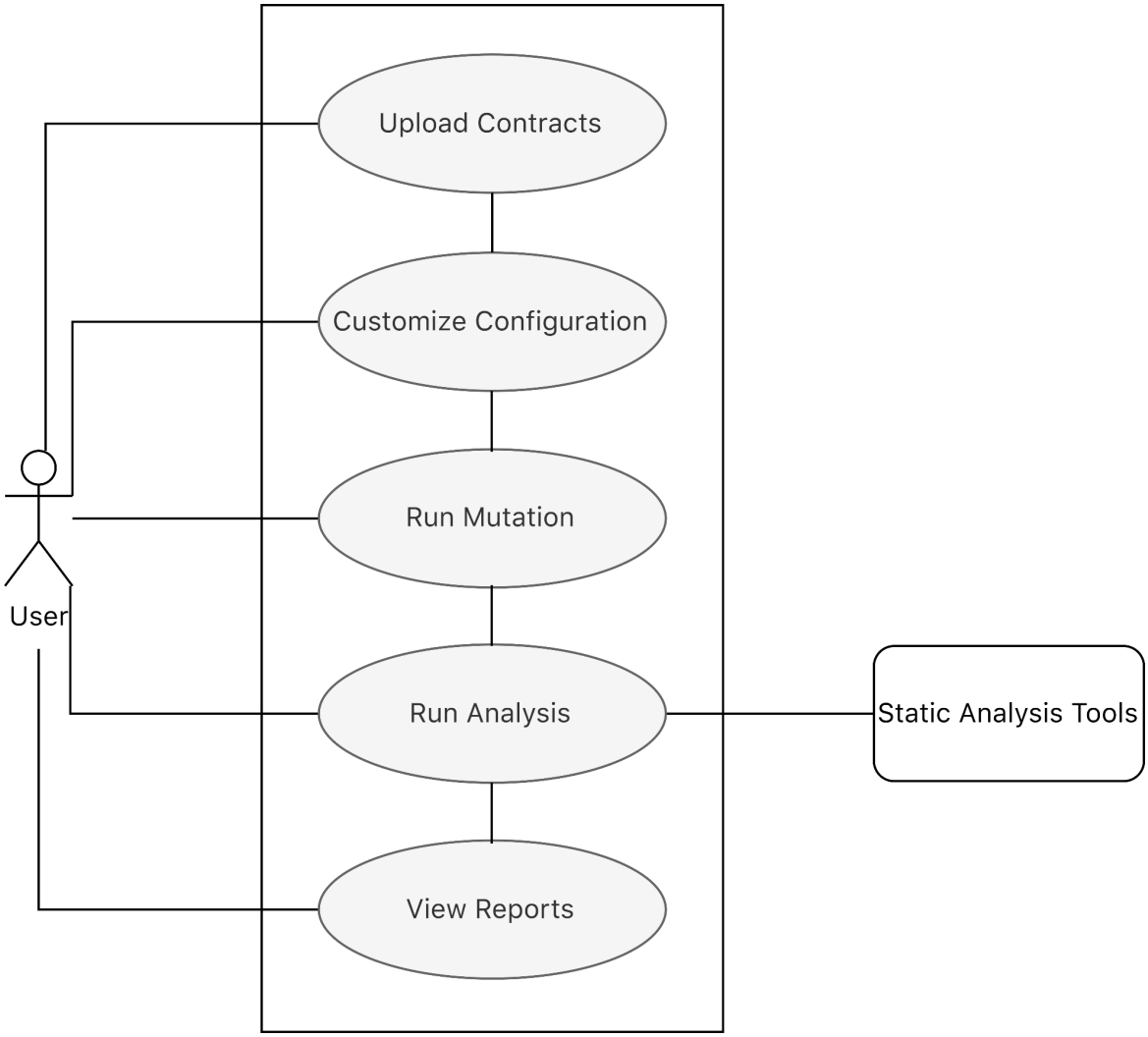


Fig 3: Use Case Diagram Level 1

Description of Use Case Diagram Level 1:

1. **Upload Contracts**

The user uploads one or more Solidity smart contracts or a directory containing contracts for analysis. This step prepares the contracts for mutation and evaluation.

2. **Customize Configuration**

The user specifies the types of vulnerabilities to inject into the contracts. This configuration tailors the mutation process to the user's needs.

3. Run Mutation

The user initiates the mutation process. MASS parses the contracts into Abstract Syntax Trees (ASTs) and identifies suitable locations for injecting the specified vulnerabilities. The process ensures that the injected mutations are both syntactically valid and semantically meaningful.

4. Run Analysis

MASS automatically runs predefined static analysis tools on the mutated contracts. These tools evaluate the contracts, producing results such as detected vulnerabilities, false positives, and false negatives.

5. View Reports

The user accesses detailed reports generated by MASS. The reports include:

- A list of vulnerabilities detected and missed by each tool.
- Comparative performance metrics such as detection rates and analysis times.
- Highlight the potential impact of undetected vulnerabilities on a smart contract.

4. Data-based Modeling

No kind of database or data storage system is used for this tool So this section does not contain any relevant analysis.

5. Class-based Modeling

This chapter describes the class-based modeling of MASS. Class-based modeling represents the objects that the system will manipulate, the operations (also called methods or services) that will be applied to the objects to affect the manipulation, relationships (some hierarchical) between the objects, and the collaborations that occur between the classes that are defined. The elements of a class-based model include classes and objects, attributes, operations, class-responsibility-collaborator (CRC) models, collaboration diagrams, and packages.

5.1 Analysis Classes

After identifying the nouns from the scenario, I filtered nouns belonging to the solution domain using General Classification (External entities, Things, Events, Roles, Organizational units, Places, and Structures). Nouns selected as a potential class were filtered using Selection Criteria (Retained information, Needed services, Multiple attributes, Common attributes, Common operations, and Essential requirements) [7]. After performing analysis on potential classes, I have found the following analysis classes -

1. ContractParser
2. AST (Abstract Syntax Tree)
3. Configuration
4. MutationEngine
5. VulnerabilityInjectionEngine
6. ToolRunner
7. ResultInspector
8. ReportGenerator

5.2 Class Cards

Table 1: CRC Card of ContractParser.

ContractParser	
Attributes	**Methods**
- source_files - parsing_options	+ parse_contracts() + read_source() + generate_ast()
Responsibilities	**Collaborators**
• Reads and parses Solidity smart contract source code files provided by the user. • Converts the parsed source code into an AST for further analysis and manipulation.	• Configuration • AST

Table 2: CRC Card of AST.

AST	
Attributes	**Methods**
- root_node - nodes - contract_name - metadata	+ traverse() + find_nodes() + insert_nodes() + delete_nodes() + replace_node() + to_source_code()
Responsibilities	**Collaborators**
• Holds the hierarchical structure of the parsed smart contract code. • Offers methods to traverse, query, and manipulate the AST nodes. • Holds annotation for bug injection.	• ContractParser • MutationEngine • VulnerabilityInjectionEngine

Table 3: CRC Card of Configuration.

Configuration	
Attributes	**Methods**
- vulnerabilities_to_inject - analysis_tools - output_options - parsing_options - mutation_options - tool_settings - report_settings	+ get_settings() + set_settings() – validate_settings()
Responsibilities	**Collaborators**
• Maintains user-defined settings such as selected vulnerabilities to inject, tool preferences, and output options. • Offers interfaces for other classes to retrieve configuration parameters. • Supplies default settings when user input is not provided.	• MutationEngine • ToolRunner • ReportGenerator

Table 4: CRC Card of MutationEngine.

MutationEngine	
Attributes	**Methods**
- ast - config - mutation_log	+ get_mutated_ast() + set_ast() + identify_injection_points() + apply_mutations() + log_mutations()
Responsibilities	**Collaborators**
• Coordinates the overall process of mutating smart contracts. • Analyzes the AST to find syntactically valid and semantically appropriate locations for injecting vulnerabilities.	• AST • VulnerabilityInjectionEngine • Configuration

Table 5: CRC Card of VulnerabilityInjectionEngine.

VulnerabilityInjectionEngine	
Attributes	**Methods**
- vulnerability_templates - template_paths - ast - injection_context - injected_vulnerabilities	+ inject_vulerabilities() – get_template() + validate_injection() + record_injections() + set_ast()
Responsibilities	**Collaborators**
• Handles bug injection into annotated locations.	• MutationEngine • AST • Configuration

Table 6: CRC Card of ToolRunner.

ToolRunner	
Attributes	**Methods**
- analysis_tools - mutated_contracts - tool_results - execution_log - tool_settings	+ execute_tools() – run_command() + collect_results() – handle_errors() – parse_tool_output()
Responsibilities	**Collaborators**
• Automates the running of predefined static analysis tools on the mutated contracts. • Gather output data from each analysis tool for further inspection.	• Configuration • ResultInspector

Table 7: CRC Card of ResultInspector.

ResultInspector	
Attributes	**Methods**
- tool_results - injected_vulnerabilities - analysis_data - false_positives - false_negatives - detection_rates	+ analyze_results() – identify_false_positives() – identify_false_negatives() – calculate_detection_rates() – prepare_summary() – set_tool_results() – set_injected_vulerabilities()
Responsibilities	**Collaborators**
● Interprets the results from static analysis tools to identify detected vulnerabilities. ● Compares tool findings against annotated AST to determine false positives and false negatives. ● Prepares data on each tool's performance for reporting.	● ToolRunner ● ReportGenerator

Table 8: CRC Card of ReportGenerator.

ReportGenerator	
Attributes	**Methods**
- analysis_data - report_settings - report_content - output_destination	+ generate_report() – format_report() – include_metrics() – highlight_findings() – set_analysis_data()
Responsibilities	**Collaborators**
● Creates reports detailing the analysis results. ● Incorporates performance metrics such as detection rates and analysis times. ● Emphasizes false negatives for user review.	● ResultInspector ● Configuration

5.3 Class-Responsibility-Collaborator Diagram

The CRC Diagram for MASS is given below [6].

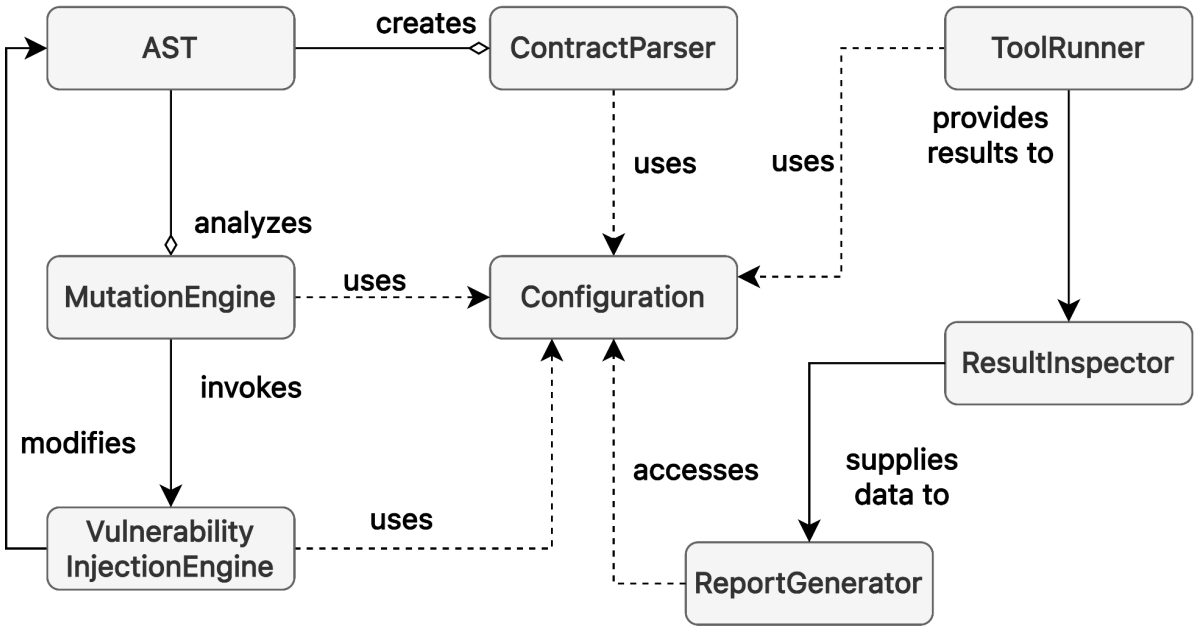


Fig 4: CRC Diagram of MASS.

6. Software Design Architecture

Architectural design is a visual representation in software engineering that outlines the high-level structure and organization of a software system. It focuses on defining the major components or modules of the system, their interactions, and the overall system's architecture.

6.1 Architectural Context Design

At the architectural design level, a software architect uses an architectural context diagram (ACD) to model the manner in which software interacts with entities external to its boundaries [7]. Systems that interoperate with the target system are represented as -

- **Superordinate systems—** Those systems that use the target system as part of some higher-level processing scheme.
- **Subordinate systems—** Those systems that are used by the target system and provide data or processing that are necessary to complete target system functionality.
- **Peer-level systems—** Those systems that interact on a peer-to-peer basis (i.e., information is either produced or consumed by the peers and the target system).
- **Actors—** Entities (people, devices) that interact with the target system by producing or consuming information that is necessary for requisite processing.

The architectural context diagram for MASS is given below.

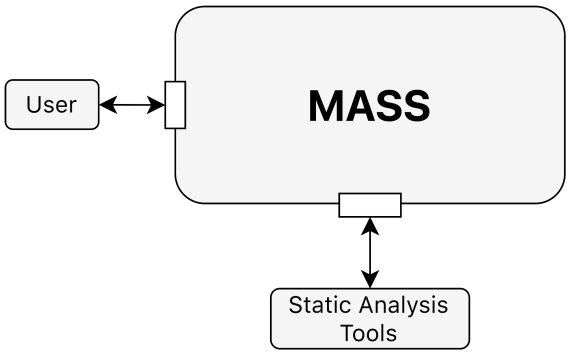


Fig 5: Architectural Context Diagram of MASS.

Representation of MASS in Architectural Context Diagram:

- **Subordinate Systems—*Static Analysis Tools.*** The contracts are analyzed via different static analysis tools.
- **Actor—*Users.*** The primary actors use MASS to mutate smart contracts, run static analysis, and evaluate the reports. They provide configurations, view reports and analyze results.

6.2 Top Level Components

Overall architectural structure with top level components is illustrated below. Note that, Top Level Components diagrams show a very high level view of our system. It is further refined into architecture in the following sections. We keep the top level diagrams to 3 independent responsibilities due to the inherent smaller size of the system and for ease of understanding the system at higher level.

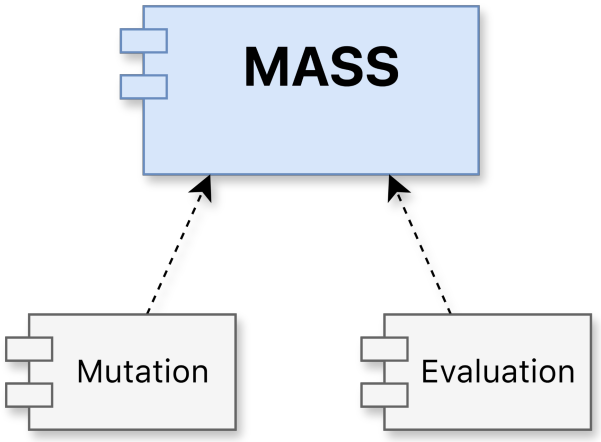


Fig 6: Top Level Components of MASS.

6.3 Instantiation of the Architecture

The top level components are refined into smaller components. We also apply architecture on the components to make a maintainable collection of components. The architecture is applied to a specific problem with the intent of demonstrating that the structure and components are appropriate.

Component: Mutation

For the Mutation component, we opted to use the Pipe and Filter architecture. The Pipe and Filter architecture is a software design pattern that organizes a system as a series of processing elements (filters) connected by communication channels (pipes). It is particularly useful for designing applications that involve processing streams of data in a linear and sequential manner. Each filter performs a specific task on the incoming data and passes the results to the next filter through the pipes, creating a data flow pipeline.

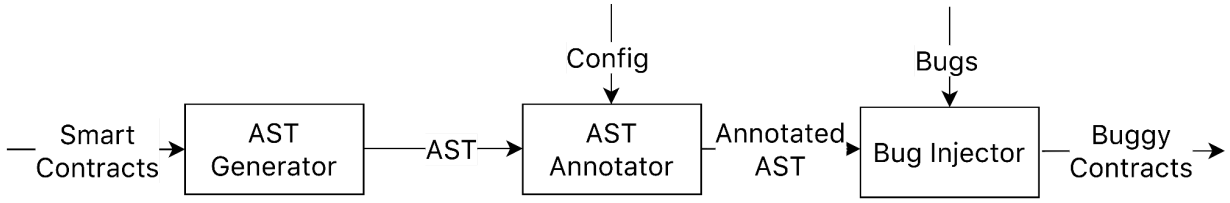


Fig 7: Architecture Instantiation on Component '*Mutation*'.

Component: Evaluation

For the Evaluation component also, we opted to use the Pipe and Filter architecture.

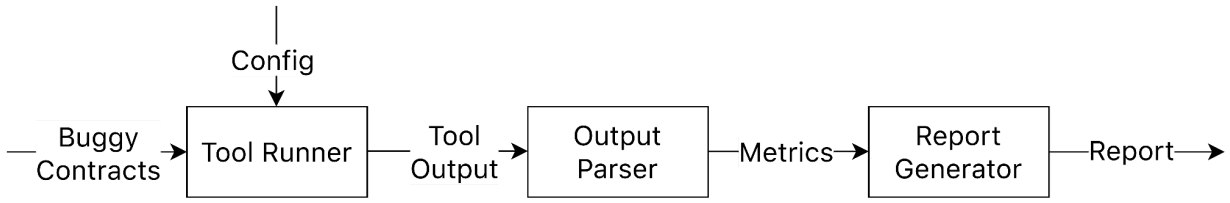


Fig 8: Architecture Instantiation on Component '*Evaluation*'.

6.4 Mapping Requirements to Software Architecture

The mapping of the derived components with the requirements can be shown as below.

Table 9: Mapping of Components with Requirements.

Requirements	**Components**
Users can configure MASS to select which bugs to inject into the smart contracts.	Mutation, Evaluation
Users can view the report generated by MASS after analyzing the contracts with static analysis tools.	Evaluation
MASS will automatically analyze the source code, inject bugs and analyze the contracts with the static analysis tools.	Mutation, Evaluation
Report will contain false positives and false negatives reported by each of the tools.	Evaluation
Report will contain comparative analysis of performance metrics of the tools.	Evaluation
Users could receive insights into how injected vulnerabilities impact gas costs.	Evaluation

7. Preliminary Test Plan

In this chapter, a high-level description of testing goals and a summary of features to be tested are presented.

7.1 High-Level Description of Testing Goals

The testing goals for MASS are as follows.

- To demonstrate that the tool meets its functional and non-functional requirements.
- To ensure that the injected mutations are both syntactically valid and semantically meaningful.
- To verify that the tool integrates seamlessly with the static analysis tools.
- To verify that the tool generates accurate false positive and false negative reports.
- To validate the correctness and clarity of the detailed analysis reports generated by the tool.
- To confirm that the tool handles user-provided configurations and inputs appropriately.
- To verify the performance and scalability of the tool during mutation.

7.2 Test Cases

Test Case 1: Mutation Syntactic Validity

- **Description:** Verify that mutated contracts compile successfully without syntax errors.
- **Preconditions:**
    - A valid smart contract is available for mutation.
- **Test Steps:**
    - Upload the valid smart contract to the tool.
    - Select vulnerabilities to inject (e.g., reentrancy, unchecked send).
    - Run the mutation process to generate mutated contracts.
    - Attempt to compile the mutated contracts using a Solidity compiler.
- **Expected Outcomes:**

    - Mutated contracts compile without syntax errors.
    - The compiler does not report any new syntax issues introduced by mutations.

Test Case 2: Mutation Semantic Validity

- **Description:** Verify that injected vulnerabilities are functionally present and can be exploited.
- **Preconditions:**
    - A test environment is set up.
    - Tools for interacting with smart contracts are available.
- **Test Steps:**
    - Deploy the mutated smart contract to the test environment.
    - Attempt to exploit the injected vulnerabilities using appropriate scripts or tools.
- **Expected Outcomes:**
    - The injected vulnerabilities behave as expected.
    - The exploit attempts are successful, demonstrating that the mutations are semantically meaningful.

Test Case 3: Static Analysis Tool Integration

- **Description:** Verify that the tool can execute all predefined static analysis tools on mutated contracts.
- **Preconditions:**
    - All required static analysis tools are installed and configured.
    - Mutated contracts are available.
- **Test Steps:**
    - Configure the tool to use all supported static analysis tools.
    - Run the analysis process on the mutated contracts.
- **Expected Outcomes:**
    - All analysis tools are invoked successfully.

    - Analysis results are collected without errors.
    - The tool handles the output from each analysis tool correctly.

Test Case 4: Handling Analysis Tool Failures

- **Description:** Verify that the tool handles failures of any static analysis tool.
- **Preconditions:**
    - One of the static analysis tools is intentionally misconfigured or unavailable.
- **Test Steps:**
    - Run the analysis process with the misconfigured tool included.
- **Expected Outcomes:**
    - The tool reports an error specific to the misconfigured analysis tool.
    - The overall analysis process continues with the remaining tools.
    - The final report indicates the failure of the specific tool without affecting other results.

Test Case 5: Detection of False Negatives

- **Description:** Verify that the tool correctly identifies vulnerabilities that were injected but not detected by analysis tools (false negatives).
- **Preconditions:**
    - A mutated contract with known injected vulnerabilities.
- **Test Steps:**
    - Run the analysis process on the mutated contract.
    - Review the generated report for missed vulnerabilities.
- **Expected Outcomes**:
    - The report accurately lists any injected vulnerabilities that were not detected by the analysis tools.
    - The tool calculates and displays the false-negative rate.

Test Case 6: Detection of False Positives

- **Description:** Verify that the tool correctly identifies vulnerabilities reported by analysis tools that were not injected (false positives).
- **Preconditions:**
    - A clean contract with no injected vulnerabilities.
- **Test Steps:**
    - Run the analysis process on the clean contract.
    - Review the generated report for incorrectly reported vulnerabilities.
- **Expected Outcomes:**
    - The report accurately lists any vulnerabilities reported by analysis tools that were not present.
    - The tool calculates and displays the false-positive rate.

Test Case 7: Report Accuracy Verification

- **Description:** Verify that the analysis report accurately reflects the results of the analysis tools and injected mutations.
- **Preconditions:**
    - Analysis has been performed on mutated contracts.
- **Test Steps:**
    - Generate the analysis report.
    - Manually cross-reference the report with known vulnerabilities and analysis tool outputs.
- **Expected Outcomes:**
    - The report correctly lists all detected vulnerabilities.
    - All injected vulnerabilities are accounted for in the report.
    - There are no discrepancies between the report and actual analysis results.

Test Case 8: Handling of Valid User Configurations

- **Description:** Verify that the tool correctly applies user-specified configurations for vulnerability injection and analysis tools.
- **Preconditions:**

    - User specifies custom configurations (e.g., selects specific vulnerabilities and analysis tools).
- **Test Steps:**
    - Input custom configurations into the tool.
    - Run the analysis process.
    - Verify that the tool applies these configurations.
- **Expected Results:**
    - Only the selected vulnerabilities are injected.
    - Only the specified analysis tools are used.
    - The tool behaves according to the user's configurations.

Test Case 9: Handling of Invalid User Inputs

- **Description:** Verify that the tool appropriately handles invalid inputs and provides meaningful error messages.
- **Preconditions:**
    - User attempts to upload an unsupported file type or provides invalid configurations.
- **Test Steps:**
    - Attempt to upload an invalid file (e.g., a PDF document).
    - Input invalid configuration settings (e.g., select a non-existent vulnerability).
    - Run the analysis process.
- **Expected Outcomes:**
    - The tool rejects invalid inputs with clear error messages.
    - The user is informed about the nature of the error and how to correct it.
    - The tool does not crash or exhibit unexpected behavior.

Test Case 10: Performance Under Heavy Load

- **Description:** Assess the tool's performance when processing a large number of smart contracts simultaneously.
- **Preconditions:**

    - A large dataset of valid smart contracts is available (e.g., 1000 contracts).
- **Test Steps:**
    - Upload the large dataset to the tool.
    - Configure the tool with standard settings.
    - Initiate the mutation and analysis process.
    - Monitor system performance (CPU, memory usage).
- **Expected Results:**
    - The tool processes all contracts without crashing.
    - System resources are utilized efficiently.
    - Processing time remains within acceptable limits.

Test Case 11: Scalability Testing with Increasing Workloads

- **Description:** Verify that the tool scales appropriately with increasing workloads.
- **Preconditions:**
    - Multiple datasets of varying sizes are available.
- **Test Steps:**
    - Run the tool on datasets of increasing size (e.g., 10, 100, 500, 1000 contracts).
    - Record processing times and resource utilization for each run.
- **Expected Results:**
    - The tool scales linearly or near-linearly with workload increases.
    - No significant degradation in performance is observed.
    - The tool remains stable and responsive throughout the tests.

8. Conclusion

In this report, I have outlined all the essential technical details necessary for the development of MASS. The scope, assumptions, and usage scenarios have been thoroughly discussed to provide a clear context for the tool's application. Scenario-based modeling, data modeling, and class-based modeling have been performed to define the system's functionality and structure meticulously. Additionally, the architectural design has been elaborated to illustrate the relationships between system components and their interactions with external systems.

A high-level description of the testing goals, along with a summary of the features and items to be tested, has been presented to ensure the reliability and effectiveness of MASS. Figures and tables have been included throughout the report to enhance the reader's understanding of the technical aspects of the tool. While MASS serves as a comprehensive solution for injecting vulnerabilities into smart contracts and evaluating the performance of static analysis tools, there are opportunities for further enhancements. Future scopes include integrating additional static analysis tools, expanding the range of vulnerabilities that can be injected, and incorporating advanced analysis techniques for more precise and adaptive assessments.

This document provides a clear overview of MASS's workflow, ensuring the reader gains a comprehensive understanding of the technical design and functionality of the system. It is hoped that this report will serve as a valuable resource for stakeholders and developers working on the tool.

References

[1] Asem Ghaleb and Karthik Pattabiraman. 2020. How Effective Are Smart Contract Analysis Tools? Evaluating Smart Contract Static Analysis Tools using Bug Injection. In *Proceedings of the 29th ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA '20), July 18-22, 2020, Virtual Event*, USA. ACM, New York, NY, USA, 13 pages.

[2] Amit Seal Ami, Syed Yusuf Ahmed, Radowan Mahmud Redoy, Nathan Cooper, Kaushal Kafle, Kevin Moran, Denys Poshyvanyk, and Adwait Nadkarni. 2023. MASC: A Tool for Mutation-Based Evaluation of Static Crypto APIMisuseDetectors. In *Proceedings of the 31st ACMJoint EuropeanSoftware Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE '23), December 3–9, 2023, San Francisco*, CA, USA. ACM, NewYork, NY, USA, 5 pages.

[3] Feist, J., Grieco, G., & Groce, A. (2019, May). Slither: a static analysis framework for smart contracts. In 2019 IEEE/ACM 2nd International Workshop on Emerging Trends in Software Engineering for Blockchain (WETSEB) (pp. 8-15). IEEE

[4] Cyfrin Official Documentation. Available at: https://docs.cyfrin.io/ [Accessed Nov. 05, 2024]

[5] History of Ethereum Security Vulnerabilities, Hacks, and Their Fixes. Available at: https://applicature.com/blog/blockchain-technology/history-of-ethereum-security-vulnerabilities-hacks-and-their-fixes [Accessed Oct. 06, 2024]

[6] Javatpoint. UML Association vs Aggregation vs Composition. Available at: https://www.javatpoint.com/uml-association-vs-aggregation-vs-composition [Accessed Nov. 26, 2024].

[7] Pressman, R. S., & Maxim, B. R. (2019). *Software engineering: A practitioner's approach* (9th ed.). McGraw-Hill Education.

[8] Mythril Official Documentation. Available at: https://mythril-classic.readthedocs.io [Accessed Oct. 06, 2024]